

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 02 -

# REFATORAÇÃO DE CÓDIGO E QUALIDADE EM ML

## SUMÁRIO

|                                  |    |
|----------------------------------|----|
| O QUE VEM POR AÍ? .....          | 3  |
| HANDS ON .....                   | 4  |
| SAIBA MAIS.....                  | 5  |
| O QUE VOCÊ VIU NESTA AULA? ..... | 17 |
| REFERÊNCIAS.....                 | 19 |

EMSE

## O QUE VEM POR AÍ?

Em projetos de Machine Learning, não basta que o modelo funcione ou que as métricas sejam boas em um primeiro momento. À medida que os dados mudam, os experimentos se acumulam e o projeto cresce, a forma como o código é escrito passa a influenciar diretamente a confiabilidade dos resultados e a capacidade de evolução do sistema. Esta aula convida você a olhar além dos algoritmos e métricas, explorando como a qualidade do código impacta todo o ciclo de vida de soluções baseadas em dados.

Ao longo deste material, você aprenderá a reconhecer sinais de degradação no código, entender quando e por que refatorar e como aplicar melhorias de forma segura e incremental. Mais do que boas práticas de programação, o foco aqui é desenvolver um olhar crítico sobre pipelines de Machine Learning, garantindo que eles sejam claros, reproduzíveis e confiáveis ao longo do tempo — habilidades essenciais para quem deseja atuar de forma profissional e responsável na área de dados.

## HANDS ON

Nesta aula, discutiremos a refatoração como uma prática essencial para a manutenção e a evolução de projetos de Machine Learning. Diferentemente de correções de erros ou otimizações de desempenho, a refatoração atua na organização interna do código, buscando torná-lo mais claro, legível e preparado para mudanças futuras, sem alterar o comportamento do modelo ou seus resultados estatísticos.

Analisaremos por que projetos de ML envelhecem rapidamente, destacando fatores como mudanças nos dados, pressão por métricas, acúmulo de experimentos e transição de códigos exploratórios para produção. Nesse cenário, problemas estruturais surgem com frequência e nem sempre aparecem como falhas de execução, mas afetam diretamente a confiabilidade e a reprodutibilidade do pipeline.

Introduziremos o conceito de code smell como um sinal de alerta para identificar esses problemas, abordando exemplos comuns em ML, como funções muito longas, duplicação de código e uso excessivo de variáveis globais. Discutiremos como esses padrões dificultam a compreensão do fluxo de dados, aumentam o risco de inconsistências estatísticas e tornam o projeto mais frágil ao longo do tempo.

Por fim, apresentaremos a refatoração guiada pela Regra dos Escoteiros e o conceito de refatoração segura, mostrando como melhorias incrementais e bem controladas permitem evoluir o código sem quebrar o comportamento do modelo. Qualidade de código e qualidade estatística caminham juntas, sendo fundamentais para o desenvolvimento de soluções de Machine Learning confiáveis e sustentáveis.

## SAIBA MAIS

Em projetos de Machine Learning, o código raramente nasce “pronto”. Ele surge de experimentos, hipóteses, testes rápidos e iterações constantes. O(a) cientista de dados explora variáveis, testa modelos, ajusta métricas e valida resultados estatísticos em um ambiente naturalmente incerto. Nesse cenário, escrever código rápido é quase uma exigência — afinal, o objetivo inicial é aprender com os dados.

O problema começa quando esse código experimental deixa de ser apenas um teste e passa a sustentar decisões reais. Modelos que antes eram protótipos passam a rodar diariamente, alimentar dashboards, impactar clientes e orientar estratégias de negócio. No entanto, o código que foi escrito para explorar hipóteses nem sempre está preparado para ser mantido, entendido ou evoluído ao longo do tempo.

Além disso, projetos de Machine Learning envelhecem rapidamente. A distribuição dos dados muda, novas variáveis são incorporadas, métricas precisam ser revistas e modelos precisam ser reavaliados. Quando o código está confuso, duplicado ou mal organizado, qualquer alteração simples se transforma em um risco: um pequeno ajuste pode quebrar o pipeline, gerar inconsistências estatísticas ou produzir resultados difíceis de reproduzir.

Nesse contexto, a falta de organização no código deixa de ser apenas um problema estético e passa a afetar diretamente a confiabilidade do modelo. Pré-processamentos duplicados podem gerar versões diferentes do mesmo dado, métricas calculadas de formas distintas comprometem comparações entre experimentos e pipelines pouco claros dificultam a identificação de erros como vazamento de dados ou validações incorretas.

É nesse ponto que a refatoração se torna essencial. Refatorar não significa “reescrever tudo” ou “mudar o modelo”, mas sim reorganizar o código para que ele reflita melhor a lógica do problema, seja mais legível, mais modular e mais confiável. Trata-se de transformar código que “funciona hoje” em código que continuará funcionando amanhã — mesmo com novos dados, novas pessoas no time e novas perguntas de negócio.

Portanto, antes de falar sobre algoritmos, métricas ou otimizações, é fundamental entender que qualidade de código é parte da qualidade científica em

Machine Learning. Um modelo só é tão confiável quanto o processo que o constrói. Refatoração é o elo entre experimentação rápida e sistemas analíticos robustos, permitindo que projetos de ML cresçam sem acumular erros, inconsistências e dívidas técnicas invisíveis.

### O que é refatoração?

Refatoração é o processo de melhorar a estrutura interna do código sem alterar seu comportamento externo. Ou seja, o modelo continua entregando as mesmas previsões, com os mesmos resultados, mas o código fica mais legível, organizado, testável e fácil de manter.

Em projetos de Machine Learning, refatorar significa, por exemplo:

- Organizar melhor pipelines de dados.
- Separar responsabilidades (leitura de dados, pré-processamento, modelagem, avaliação).
- Eliminar duplicações de código.
- Melhorar nomes de variáveis, funções e classes.
- Tornar o fluxo mais claro para quem vai manter ou evoluir o projeto.

Embora refatoração, correção de erros (bug fix) e otimização apareçam frequentemente lado a lado no dia a dia de projetos de Machine Learning, eles atendem a objetivos distintos e não devem ser confundidos. Em um primeiro momento, pode ser necessário corrigir um comportamento incorreto do código, garantindo que o modelo produza resultados estatisticamente válidos; em outros casos, o foco está em tornar o treinamento ou a inferência mais eficiente do ponto de vista computacional.

A refatoração, por sua vez, ocupa um espaço diferente: ela não busca corrigir resultados errados nem tornar o sistema mais rápido, mas sim melhorar a estrutura interna do código, tornando-o mais claro, organizado e confiável para evolução futura. Compreender essa diferença é fundamental para manter projetos de ML saudáveis ao longo do tempo, evitando que soluções rápidas se transformem em dívidas técnicas

que comprometem tanto a manutenção do código quanto a confiabilidade das análises estatísticas.

### **Qual diferença entre Bug Fix, Otimização e Refatoração?**

Esses três conceitos são frequentemente confundidos, mas têm objetivos bem diferentes.

Um bug fix acontece quando existe um erro funcional no sistema.

Exemplos em ML:

- O modelo está usando dados de teste no treino (data leakage).
- Um cálculo estatístico está errado.
- Uma variável foi codificada incorretamente.
- Um script quebra ao rodar em produção.

O comportamento está errado, então o bug fix corrige o resultado.

Já a otimização busca melhorar performance, como:

- Reduzir o tempo de treinamento.
- Diminuir o uso de memória.
- Tornar a inferência mais rápida.
- Escalar melhor o pipeline.

Exemplos:

- Trocar loops por operações vetorizadas.
- Ajustar hiperparâmetros para acelerar convergência.
- Usar estruturas de dados mais eficientes.

O resultado final pode ser o mesmo, mas o foco é velocidade, custo ou eficiência computacional.

Por fim, a refatoração foca exclusivamente em qualidade de código.

Exemplos:

- Quebrar uma função de 300 linhas em várias funções menores.
- Criar uma função única para pré-processamento que antes estava duplicado.
- Renomear variáveis confusas.
- Organizar o projeto em módulos claros.

O modelo continua produzindo exatamente as mesmas saídas, mas o código fica mais limpo, legível, fácil de testar e confiável para evoluir.

A distinção entre bug fix, otimização e refatoração torna-se ainda mais relevante quando observamos como projetos de Machine Learning envelhecem rapidamente. Diferentemente de sistemas tradicionais, modelos de ML dependem de dados que mudam ao longo do tempo, de hipóteses estatísticas que precisam ser reavaliadas e de regras de negócio que estão em constante evolução. Nesse cenário, erros que antes não existiam passam a surgir, exigindo correções frequentes; volumes maiores de dados demandam otimizações para manter desempenho; e pipelines originalmente experimentais precisam ser reorganizados para suportar novas versões do modelo.

Quando essas três frentes não são claramente separadas, o projeto se torna frágil: correções emergenciais se misturam com melhorias estruturais, código experimental vira produção sem organização adequada e cada nova alteração aumenta o risco de introduzir inconsistências estatísticas. Assim, a ausência de refatoração contínua acelera o envelhecimento do projeto, transformando mudanças naturais do ciclo de vida do Machine Learning em fontes constantes de retrabalho, instabilidade e perda de confiança nos resultados do modelo.

### **Por que projetos de Machine Learning envelhecem rápido**

Projetos de ML envelhecem mais rápido do que software tradicional porque dependem fortemente de dados e do contexto do negócio, que mudam constantemente.

- Mudança na distribuição dos dados (Data Drift).
  - O comportamento dos usuários muda.



- O mercado muda.
- O padrão estatístico dos dados deixa de ser o mesmo.

Um código mal estruturado dificulta:

- Reanálise exploratória.
- Retreinamento.
- Comparação entre versões do modelo.
- Código experimental vira produção

É muito comum:

- Código feito “rápido” para testar hipóteses.
- Scripts exploratórios serem reaproveitados.
- Falta de padronização e testes.

Sem refatoração, o projeto vira um emaranhado de scripts difíceis de entender.

- Troca de pessoas no time

Em ML, projetos vivem mais do que as pessoas que os criaram.

- Novo(a) cientista de dados precisa entender o pipeline.
- Código confuso gera erros, retrabalho e desconfiança.
- Evolução constante do modelo

Modelos não são estáticos:

- Novas features.
- Novos algoritmos.
- Novas métricas.
- Novas regras de negócio.

Sem refatoração contínua, cada mudança aumenta o risco de quebrar algo.

Uma vez compreendida a importância da refatoração e sua relação com a correção de erros, a otimização e o envelhecimento acelerado de projetos de Machine Learning, surge uma pergunta natural: como identificar, na prática, que um código

precisa ser refatorado? Nem sempre os problemas aparecem como falhas explícitas ou erros de execução; muitas vezes, o código “funciona”, o modelo treina e as métricas parecem aceitáveis. No entanto, sinais sutis indicam que algo não vai bem — trechos difíceis de entender, funções longas, duplicação de lógica e dependências pouco claras.

Esses sinais são conhecidos como code smells. Eles não representam, necessariamente, um erro imediato, mas revelam decisões de implementação que tornam o projeto frágil e difícil de manter. Em Machine Learning, esses smells surgem com frequência porque códigos exploratórios acabam sendo promovidos a produção, há pressão constante por melhorar métricas rapidamente e inúmeros experimentos vão se acumulando sem organização. Reconhecer esses padrões é o primeiro passo para refatorar de forma consciente e preservar a qualidade do código e a confiabilidade dos modelos ao longo do tempo.

## Conceito de Code Smell

Code smell é um termo utilizado para descrever características no código que não são necessariamente erros, mas indicam que a implementação pode ser ineficiente, difícil de entender ou de manter. Essas características podem ser sinais de que o código precisa de melhorias, refatoração ou revisão para garantir sua qualidade a longo prazo.

Exemplos de code smells comuns incluem:

- **Funções/métodos muito longos:** funções que tentam fazer muitas coisas ao mesmo tempo, tornando-se difíceis de entender e de manter.
- **Duplicação de código:** blocos de código semelhantes espalhados por várias partes do projeto.
- **Nomes de variáveis ou funções confusos:** nomes que não representam claramente o que a função ou variável faz, dificultando a leitura do código.
- **Dependências não explícitas:** quando o fluxo do código não é claro, como por exemplo ao usar variáveis globais que afetam o comportamento de várias partes do código sem que isso fique claro.

É importante entender que code smell não é a mesma coisa que um erro de execução. Um código com code smell pode funcionar sem falhas aparentes e até retornar resultados válidos, mas pode ser difícil de entender, testar ou modificar no futuro.

Erro de execução ocorre quando há algo no código que impede o funcionamento do programa, como um erro de sintaxe, falha ao acessar uma variável não inicializada ou outro tipo de problema que impede a execução.

Exemplo:

- Erro de execução: um código com erro de sintaxe, como esquecer de fechar um parêntese ou utilizar uma variável não definida.
- Code smell: mesmo sem erro de execução, o código pode ser confuso ou difícil de modificar.

Em projetos de Machine Learning, é muito comum que code smells apareçam devido à natureza dos experimentos e à forma como o código é desenvolvido. Vamos explorar três razões principais.

**Código Exploratório Virando Produção:** em muitas situações, o código que inicialmente foi escrito para explorar dados ou testar hipóteses rapidamente acaba virando a base de produção. No entanto, códigos exploratórios muitas vezes não seguem boas práticas de desenvolvimento, como modularidade e clareza, já que o foco inicial é validar uma ideia, não a qualidade do código.

**Pressão por Métricas:** a pressão para entregar resultados rápidos e apresentar métricas de modelos de ML pode levar cientistas de dados a priorizarem a produção de métricas em vez de se preocupar com a clareza e a organização do código. Como resultado surgem code smells, como duplicação de código ou falta de clareza nas funções de cálculo das métricas.

**Muitos Experimentos Acumulados:** em projetos de ML, frequentemente são realizados muitos experimentos com variações de hiperparâmetros, modelos diferentes ou pré-processamentos diferentes. À medida que esses experimentos se acumulam, dificulta-se o controle sobre o código, e os code smells aparecem por falta de organização e modularização.

Code smells são um reflexo de como o código foi escrito e evoluiu ao longo do tempo. Embora eles não sejam erros de execução, sua presença constante pode gerar dificuldades na manutenção do projeto, aumento de riscos de erros em produção e baixa confiabilidade nos resultados do modelo. Em Machine Learning, a pressão por entregas rápidas e a constante adição de novos experimentos tornam os code smells ainda mais frequentes. Portanto, identificar e tratar esses sinais é essencial para manter a qualidade e a escalabilidade de projetos de ML.



Figura 1 - Funções muito longas como code smell em projetos de Machine Learning  
Fonte: Google Imagens (2026)

Um dos code smells mais frequentes em projetos de Machine Learning é a presença de funções excessivamente longas, que concentram múltiplas responsabilidades em um único bloco de código. Em geral, essas funções surgem de forma natural durante a fase exploratória do projeto: o(a) cientista de dados começa lendo os dados, realiza transformações, treina o modelo, avalia métricas e salva resultados — tudo dentro de uma única função ou script, pois o objetivo inicial é validar rapidamente uma hipótese.

O problema aparece quando esse código deixa de ser apenas exploratório e passa a ser reutilizado, compartilhado ou colocado em produção. Funções muito longas tornam o fluxo do pipeline difícil de entender, pois misturam etapas conceitualmente diferentes, como pré-processamento, treinamento e avaliação. Isso dificulta a leitura, aumenta o risco de erros e torna qualquer modificação futura mais custosa, já que alterar uma parte da função pode impactar outras etapas de forma inesperada.

Nesse contexto, o conceito de code smell atua como um sinal de alerta. Uma função extensa indica que o código provavelmente está violando o princípio da responsabilidade única, isto é, está “fazendo coisas demais”. Embora o código possa funcionar corretamente, esse tipo de estrutura reduz a clareza do processo analítico e compromete a confiabilidade do projeto, especialmente em Machine Learning, onde pequenas mudanças no pipeline podem alterar distribuições de dados e métricas de desempenho.

Identificar funções longas como code smell ajuda o indivíduo desenvolvedor a perceber que o problema não está no resultado imediato do modelo, mas na forma como o raciocínio estatístico e computacional foi codificado. A refatoração, nesse caso, consiste em extrair partes da função em blocos menores e bem definidos, como funções específicas para leitura de dados, transformação de variáveis, treinamento do modelo e avaliação. Essa separação torna o código mais legível, facilita testes, reduz duplicações e aumenta a segurança ao evoluir o modelo ao longo do tempo.

Já a duplicação de código ocorre quando a mesma lógica é implementada em múltiplos pontos do projeto, geralmente com pequenas variações. Em projetos de Machine Learning, esse code smell é especialmente comum em pipelines de dados e modelagem, onde etapas como pré-processamento, engenharia de features, separação de dados e cálculo de métricas acabam sendo repetidas em diferentes scripts, notebooks ou funções.

À primeira vista, a duplicação pode parecer inofensiva — afinal, o código funciona e gera métricas. No entanto, esse padrão cria um risco silencioso: o mesmo dado pode ser tratado de formas diferentes ao longo do projeto, comprometendo a consistência estatística e dificultando a comparação entre modelos e experimentos.

Em Machine Learning, em que pequenas diferenças no pré-processamento podem alterar distribuições, métricas e decisões, duplicação de código não é apenas um problema de organização — é um problema de confiabilidade do pipeline.

## Refatoração guiada pela Regra dos Escoteiros

A Regra dos Escoteiros é um princípio simples, mas extremamente poderoso, aplicado ao desenvolvimento de software e, em especial, a projetos de Machine Learning: “deixe o código um pouco melhor do que você o encontrou”.

Em vez de encarar a refatoração como uma grande tarefa a ser feita apenas em momentos específicos do projeto, a Regra dos Escoteiros propõe uma abordagem contínua e incremental. Sempre que um(a) desenvolvedor(a) ou cientista de dados mexe em uma parte do código — seja para corrigir um bug, testar um novo modelo ou ajustar uma métrica — ele(a) aproveita a oportunidade para melhorar pequenos aspectos da qualidade do código.

Projetos de Machine Learning evoluem de forma constante. Dados mudam, modelos são substituídos, métricas são revisitadas e novos experimentos são adicionados ao pipeline. Nesse contexto, esperar por uma “refatoração completa” raramente é viável. A Regra dos Escoteiros se adapta perfeitamente a essa realidade, pois permite melhorias graduais sem interromper o fluxo de experimentação.

Além disso, muitos code smells em ML — como funções longas, duplicação de código e uso excessivo de variáveis globais — surgem aos poucos, quase de forma imperceptível. Ao aplicar a Regra dos Escoteiros, esses problemas são tratados quando aparecem, antes que se transformem em dívidas técnicas difíceis de eliminar.

Seguir a Regra dos Escoteiros não significa reescrever grandes partes do sistema. Em Machine Learning, melhorias simples já fazem grande diferença, como:

- Renomear uma variável para refletir melhor seu significado estatístico.
- Extrair uma parte de uma função longa para uma função menor e mais clara.
- Centralizar um pré-processamento que estava duplicado.
- Tornar explícitas dependências que antes estavam implícitas.
- Remover comentários obsoletos que não refletem mais o comportamento do código.

Essas pequenas ações, quando repetidas ao longo do tempo, elevam significativamente a qualidade do projeto.

## Refatoração incremental e confiabilidade estatística

Um ponto fundamental é que a Regra dos Escoteiros não altera o comportamento do modelo. Assim como em qualquer refatoração bem feita, os resultados estatísticos permanecem os mesmos. O que muda é a clareza do processo que gera esses resultados.

Em Machine Learning, isso é crucial. Pipelines mais claros facilitam:

- Reprodutibilidade dos experimentos.
- Auditoria das etapas estatísticas.
- Identificação de vazamento de dados.
- Comparação justa entre modelos.

Ou seja, refatorar seguindo a Regra dos Escoteiros contribui diretamente para a confiabilidade científica do projeto.

Ignorar pequenos problemas no código é uma das principais fontes de dívida técnica. A Regra dos Escoteiros atua como um mecanismo preventivo: ao corrigir pequenos code smells continuamente, evita-se o acúmulo de problemas que, no futuro, exigiriam grandes esforços de refatoração.

A base da refatoração segura está em separar claramente estrutura de comportamento. O comportamento é o que o modelo faz: previsões, métricas, decisões. A estrutura é como o código está organizado para produzir esses resultados. Uma refatoração bem conduzida atua apenas na estrutura, preservando o comportamento.

Não é possível refatorar com segurança aquilo que não conseguimos observar. Em Machine Learning, isso significa ter métricas claras, conjuntos de validação bem definidos e resultados reproduzíveis. Antes de refatorar, o projeto deve responder perguntas simples: quais métricas estão sendo avaliadas? Em quais dados? Com quais parâmetros?

Refatorações seguras são incrementais. Em vez de alterar todo o pipeline de uma vez, o ideal é fazer pequenas melhorias, validando o comportamento a cada passo. Esse princípio se alinha diretamente à Regra dos Escoteiros: pequenas melhorias constantes reduzem riscos e aumentam a confiança no código.

Em projetos de ML, isso pode significar:

- Extrair uma função e verificar se as métricas permanecem iguais.
- Centralizar um pré-processamento e validar os resultados.
- Remover duplicações mantendo a saída do pipeline.

### **Testes e validações como rede de segurança**

Embora projetos de Machine Learning nem sempre tenham testes tradicionais, validações funcionam como testes. Comparar métricas antes e depois da refatoração é uma forma prática de garantir que o comportamento não mudou.

Além disso, funções menores e dependências explícitas facilitam a criação de testes unitários, como:

- Testar apenas o pré-processamento.
- Validar formatos e distribuições de dados.
- Garantir que o pipeline respeita separação entre treino e teste.

Refatorar de forma insegura pode introduzir erros silenciosos, como vazamento de dados ou inconsistências no pipeline. Já a refatoração segura fortalece a confiança no projeto, pois torna o fluxo mais explícito e reduz dependências ocultas.

Com o tempo, pipelines refatorados com cuidado se tornam mais fáceis de explicar, de auditar, de evoluir e mais confiáveis estatisticamente.



## O QUE VOCÊ VIU NESTA AULA?

Ao longo desta aula, exploramos a refatoração como um elemento central para a construção de projetos de Machine Learning sustentáveis, confiáveis e preparados para evoluir ao longo do tempo. Diferentemente de uma visão limitada focada apenas em algoritmos e métricas, o conteúdo apresentado destacou que a qualidade de um projeto de ML está diretamente ligada à forma como o código organiza o raciocínio estatístico, o fluxo de dados e as decisões de modelagem.

Iniciamos a discussão contextualizando o ambiente típico de projetos de Machine Learning. Esses projetos raramente seguem um caminho linear: começam com análises exploratórias, passam por múltiplos experimentos, ajustes de variáveis, mudanças de métricas e evoluem conforme novas perguntas de negócio surgem. Nesse cenário dinâmico, é comum que códigos inicialmente escritos para explorar hipóteses acabem sendo reaproveitados em pipelines mais duradouros, muitas vezes sem a devida preocupação com organização e manutenção.

A partir desse contexto, ficou claro porque projetos de ML envelhecem rapidamente. Mudanças na distribuição dos dados, atualizações de modelos, pressão por resultados rápidos e troca de pessoas no time fazem com que pipelines frágeis se tornem difíceis de entender e arriscados de modificar.

Em seguida, diferenciamos três conceitos frequentemente confundidos no dia a dia: bug fix, otimização e refatoração. Com essa base estabelecida, introduzimos o conceito de code smell. Code smells não são erros de execução nem impedem o funcionamento imediato do código. Eles são sinais de alerta que indicam decisões de implementação que tornam o projeto mais difícil de manter, entender e evoluir. Em Machine Learning, esses sinais são especialmente importantes, pois problemas estruturais podem afetar diretamente a confiabilidade estatística, a reprodutibilidade dos experimentos e a comparação justa entre modelos.

A aula avançou então para a refatoração guiada pela Regra dos Escoteiros, que propõe uma abordagem incremental e contínua para melhorar a qualidade do código. Em vez de grandes reescritas pontuais, a regra sugere que cada modificação no projeto seja uma oportunidade para deixar o código um pouco melhor do que

estava antes. Essa prática é especialmente adequada ao contexto de Machine Learning, onde mudanças são constantes e o código está em evolução permanente.

Ao longo da aula, ficou evidente que refatoração não é apenas uma boa prática de programação, mas uma competência essencial para cientistas de dados. Código limpo e bem estruturado facilita auditorias, reduz risco de vazamento de dados, melhora a comunicação dentro do time e aumenta a confiança nas decisões baseadas em modelos. Qualidade de código e qualidade estatística caminham juntas.

Como conclusão, a aula reforça que projetos de Machine Learning bem-sucedidos não dependem apenas de bons algoritmos, mas de pipelines claros, organizados e preparados para evoluir. Identificar code smells, aplicar refatoração de forma consciente, seguir a Regra dos Escoteiros e refatorar com segurança são habilidades fundamentais para quem deseja atuar de forma profissional, responsável e sustentável na área de dados.

## REFERÊNCIAS

BECK, K. **Extreme Programming Explained**: embrace change. Boston: Addison-Wesley, 2000.

FOWLER, M. **Code smell**. 2009. Disponível em: <https://martinfowler.com/bliki/CodeSmell.html>. Acesso em: 17 mar. 2026.

FOWLER, M. **Refactoring**: improving the design of existing code. 2. ed. Boston: Addison-Wesley, 2018.

GOOGLE. **Machine Learning**: the high interest credit card of technical debt. 2015. Disponível em: <https://static.googleusercontent.com/media/research.google.com/pt-BR/pubs/archive/43146.pdf>. Acesso em: 17 mar. 2026.

MARTIN, R. C. **Clean Architecture**: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.

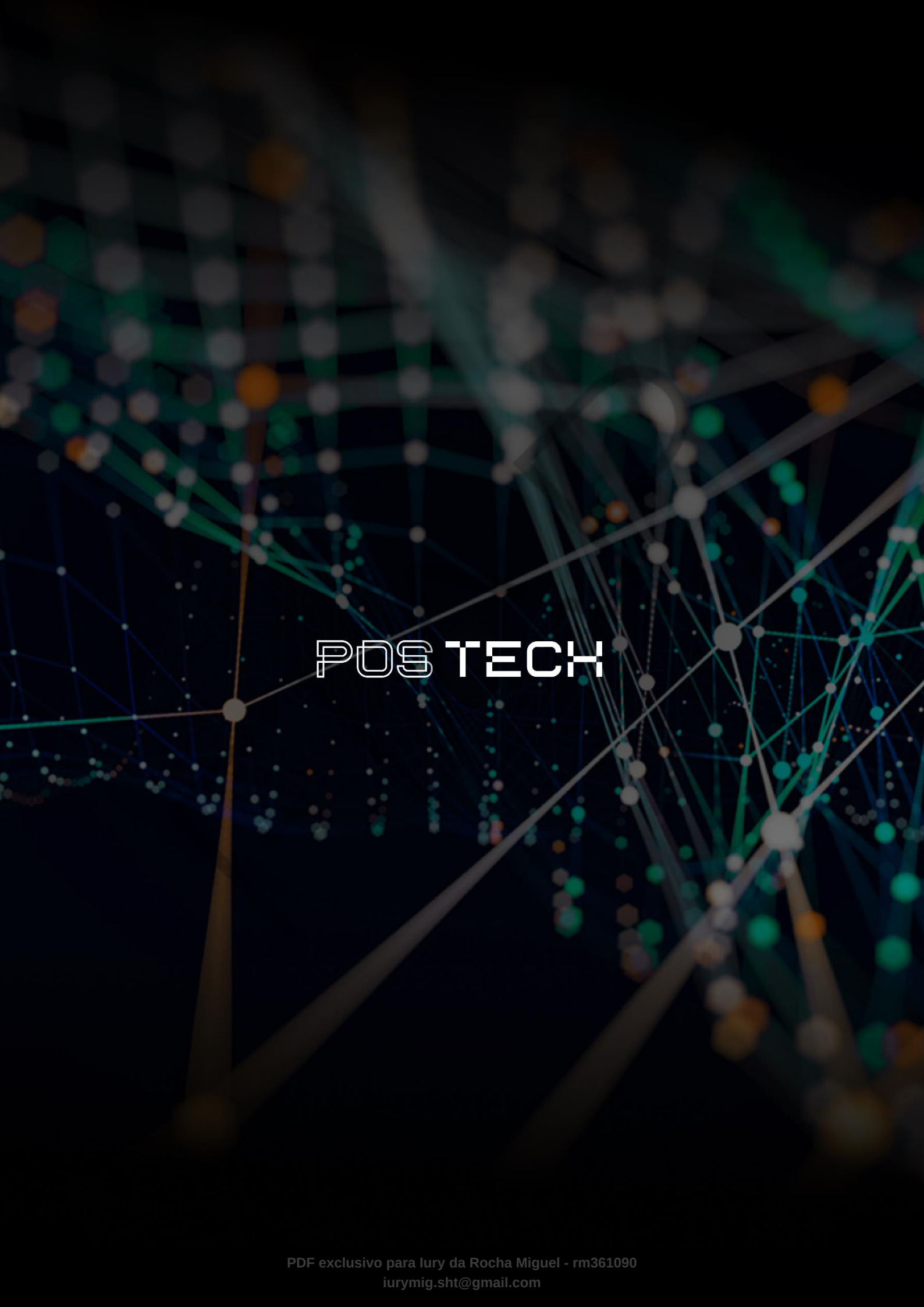
MARTIN, R. C. **Clean Code**: a handbook of agile software craftsmanship. Boston: Prentice Hall, 2008.

MICROSOFT. **MLOps**: continuous delivery and automation pipelines in machine learning. Redmond: Microsoft, 2021.

## **PALAVRAS-CHAVE**

Refatoração. Manutenibilidade. Reprodutibilidade.

EMSE



# POSTECH